

Milestone 1 Report: First Baseline — Domain-Specific RL Retrieval

Project: Domain-Specific RL Retrieval with Limited Labels (Joint RL + IR)

Team: Ali Salloum, Yazan Kbaili

Milestone: Phase 1 — Setup and first retrieval baseline

Date: March 2025

1. Background

This project targets **sample-efficient reinforcement learning for information retrieval** in high-stakes domains (legal, and optionally medical) where relevance labels are scarce. As outlined in the proposal (*Proposal_Idea4_Domain_Specific_RL_Retrieval.md*):

- **Problem:** In legal and medical IR, expert relevance judgments are expensive, general-purpose retrievers perform poorly on domain tasks, and metrics like recall@k or end-task accuracy are non-differentiable—so standard supervised learning alone is insufficient.
- **Goal:** Adapt pretrained retrievers using **policy-gradient RL** (following the ICLR 2024 framework) to optimize task-specific objectives (recall, MRR, nDCG, or downstream RAG/QA accuracy) when labels are limited.
- **Data:** We use the **CLERC** benchmark (U.S. legal case retrieval and retrieval-augmented analysis) as the primary dataset, with optional extension to LegalBench-RAG and medical benchmarks later.
- **Evaluation:** Retrieval quality is measured with **recall@k** ($k = 1, 5, 10, 100$), **MRR**, and **nDCG@10** on held-out test sets. These same metrics will later serve as rewards for the RL phase.

Establishing a reproducible **baseline pipeline**—data loading, evaluation metrics, and at least one retriever (DPR)—is the prerequisite for Phase 2 (RL adaptation) and Phase 3 (sample-efficiency ablations).

2. Theoretical Groundwork

- **IR:** Relevance and evaluation are framed in terms of standard **learning-to-rank** and the **probability ranking principle**. The metrics we use (recall@k , MRR, nDCG) are the same ones that will be used as reward signals in the RL phase, ensuring a direct link between baseline evaluation and future optimization.
- **RL (for later phases):** The planned method is **policy-gradient** optimization of a stochastic retrieval policy with respect to a reward defined from these IR metrics. Sample

efficiency will relate to exploration–exploitation and, if we use limited or noisy feedback, to **contextual bandits** or **preference-based RL**.

- **Baseline role:** A zero-shot (or domain-fine-tuned) retriever baseline provides a reference point to compare against the RL-adapted retriever and to validate that the evaluation pipeline matches the benchmark (CLERC) and proposal.

This theoretical framing was reviewed and aligned with the proposal so that implementation choices (metric definitions, data splits, reward design later) stay consistent.

3. Practical Work Completed

3.1 Environment and Data

- **Environment:** Python 3.10+ virtual environment (`.venv`) with dependencies in `setup/requirements.txt` . CLERC is loaded from HuggingFace ([jhu-clsp/CLERC](https://huggingface.co/jhu-clsp/CLERC)); no manual dataset download is required for queries and qrels.
- **Data loading:** Implemented in `setup/clerc_data.py` : loading queries (direct/indirect) and qrels (doc- or passage-level) from the official CLERC splits. Support for streaming the large passage collection (~9 GB) was added so we can run DPR on a subset of the corpus (e.g. first N passages or a small percentage) without loading the full file into memory.
- **Splits:** We use the official CLERC test queries and qrels (e.g. `queries/test.single-removed.direct.tsv` , `qrels/qrels-doc.test.direct.tsv`) for evaluation, consistent with the benchmark.

3.2 Evaluation Pipeline

- **Metrics:** In `setup/metrics.py` , we implemented `recall@k` ($k = 1, 5, 10, 100$), MRR, and `nDCG@10`. The same interface is used both for baseline reporting and (in Phase 2) for reward computation.
- **Eval API:** `setup/eval.py` provides an evaluation entry point that can take a run file (e.g. `run_dpr.json`) or a callable retriever, plus query type and qrels level, and return the same metric dictionary.

3.3 Baselines

- **DPR:** We integrated the CLERC-domain DPR model (**LegalBERT-DPR-CLERC-ft** from HuggingFace). The script `setup/run_baselines.py` encodes queries and a (sub)set of the passage collection, builds a similarity index, and retrieves top-k passages per query. To allow fast iteration without the full 9 GB collection, we support a small corpus (e.g. 2% of collection or 5k passages from a pre-encoded checkpoint), with results written to `setup/data/runs/run_dpr.json` and aggregated metrics to `setup/data/runs/baseline_metrics.json` .

- **BM25:** The pipeline supports BM25 via Pyserini; running it requires a pre-built Pyserini index from the CLERC collection (see `setup/README.md`). Mock BM25 runs are available for pipeline testing when no index is present.

3.4 First Baseline Run and Results

A first DPR baseline was run with a small subset of the corpus for speed:

```
.venv/bin/python -m setup.run_baselines --dpr --dpr-corpus-pct 0.02 --out-dir setup/data/runs
```

- **Setup:** ~5,000 passages from the collection (from checkpoint); 1,497 queries; direct queries, passage-level retrieval.
- **Output:** Metrics saved in `setup/data/runs/baseline_metrics.json`:

Metric	Value (DPR, 0.02% corpus)
recall@1	0.0
recall@5	0.0
recall@10	0.0
recall@100	~0.00067
MRR	~2.02e-05
nDCG@10	0.0

These numbers are expected to be very low because the **retrieval corpus was intentionally tiny** (~5k passages) to validate the pipeline quickly. The CLERC paper reports ~48.3% recall@1000 with zero-shot IR on the full collection; our next step is to run DPR on a larger corpus (e.g. 10k+ passages or full collection when feasible) to obtain a proper baseline before RL.

4. Division of Work (Ali Salloum & Yazan Kbaili)

Tasks for the first milestone were split as follows (kept simple and aligned with the deliverables):

Task	Owner	Description
Proposal alignment & theory	Yazan Kbaili	Reviewed proposal; ensured evaluation metrics (recall@k, MRR, nDCG) and data choices match the RL reward design and CLERC benchmark.
Data & environment	Ali Salloum	

Task	Owner	Description
		Set up Python environment, <code>requirements.txt</code> , and CLERC data loading from HuggingFace (<code>clerc_data.py</code> : queries, qrels, streaming collection).
Metrics & evaluation	Yazan Kbaili	Implemented <code>metrics.py</code> (recall@k, MRR, nDCG@10) and <code>eval.py</code> API for run files and retrievers.
Baseline runner & DPR	Ali Salloum	Implemented <code>run_baselines.py</code> (DPR with LegalBERT-DPR-CLERC-ft, optional corpus subset/checkpoint), and ran the first DPR baseline; collected <code>baseline_metrics.json</code> .
Documentation	Both	Maintained <code>setup/README.md</code> , <code>PHASE1_SETUP.md</code> , and <code>baseline_results_dev.md</code> ; this report drafted jointly.

5. Discussion

- **Achieved:** We have a working Phase 1 pipeline: CLERC data from HuggingFace, a well-defined evaluation (recall@k, MRR, nDCG@10), and a DPR baseline that runs end-to-end and writes metrics. The first run used a small corpus on purpose to verify the pipeline; results are saved and reproducible.
- **Limitations:** The current DPR numbers are not comparable to the CLERC paper because the corpus size was minimal. We need at least one full-sized baseline run (larger corpus or full collection where feasible so it will take days or even a week) before claiming a “reproduced” zero-shot baseline. BM25 baseline remains conditional on building a Pyserini index.
- **Next steps (Phase 2):** (1) Run DPR (and BM25 if index is built) on a larger corpus and record baseline metrics in `baseline_results_dev.md`. (2) Implement or adapt policy-gradient RL from the ICLR 2024 framework; plug in the same metrics as rewards. (3) Train the RL-adapted retriever on CLERC and compare to the baseline.

6. References

- **Proposal:** `Proposal_Idea4_Domain_Specific_RL_Retrieval.md`
- **CLERC:** [arXiv:2406.17186](https://arxiv.org/abs/2406.17186), [HuggingFace jhu-clsp/CLERC](https://huggingface.co/jhu-clsp/CLERC), [GitHub](https://github.com)
- **RL for retrieval (ICLR 2024):** [OpenReview](https://openreview.net/forum?id=8333333333) — policy-gradient adaptation of retrievers to task-specific goals
- **Setup and reproducibility:** `setup/README.md`, `PHASE1_SETUP.md`